Banking Multi-Agent Support Bot

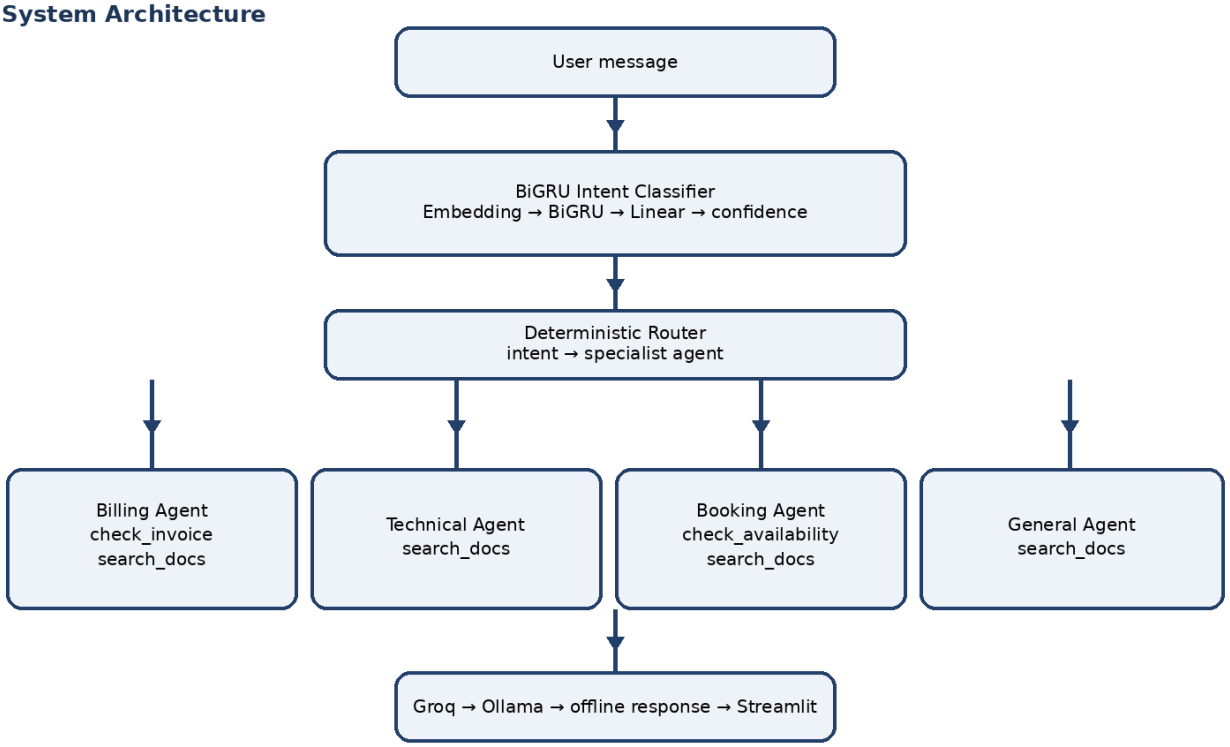
Project Documentation and Implementation Report

Domain: Banking customer support
Classifier: From-scratch Embedding + BiGRU
Orchestration: LangChain specialist agents
Interface: Streamlit

1. Project Objective

The application routes support messages to specialist agents using a trained deep-learning intent classifier. The classifier predicts one of five selected intents and produces a confidence score. A deterministic Python router maps that intent to one of four specialist agents. The selected agent then uses its own system prompt and tools to generate the response. This keeps the routing decision outside the language model.

System Architecture



2. Domain, Intents, and Dataset

Intent	Agent
card_payment_fee_charged	Billing
cash_withdrawal_charge	Billing
card_not_working	Technical
order_physical_card	Booking / card-order
country_support	General

BANKING77 is a public banking customer-service intent dataset. The selected MTEB version provides 77 labels and a 10,003-row training split with text, numeric label, and label_text fields. The project filters five intents and makes a stratified 80/10/10 split from the selected training examples.

Dataset source: https://huggingface.co/datasets/mteb/banking77

3. Deep Learning Classifier

The classifier is deliberately small and trainable from scratch. It uses a trainable word embedding, a bidirectional GRU, dropout, and a final linear classifier. Training uses Adam, cross-entropy loss, gradient clipping, and validation macro-F1 for checkpoint selection.

Classifier architecture

```python
# AI-GENERATED SECTION START
"""Small BiGRU intent classifier used by the router."""
from __future__ import annotations

import torch
from torch import nn
from torch.nn.utils.rnn import pack_padded_sequence


class IntentClassifier(nn.Module):
    def __init__(
        self,
        vocab_size: int,
        embed_dim: int = 100,
        hidden_dim: int = 128,
        num_classes: int = 5,
        pad_idx: int = 0,
        dropout: float = 0.30,
    ) -> None:
        super().__init__()
        self.embedding = nn.Embedding(vocab_size, embed_dim, padding_idx=pad_idx)
        self.gru = nn.GRU(
            input_size=embed_dim,
            hidden_size=hidden_dim,
            batch_first=True,
            bidirectional=True,
        )
        self.dropout = nn.Dropout(dropout)
        self.fc = nn.Linear(hidden_dim * 2, num_classes)

    def forward(self, x: torch.Tensor, lengths: torch.Tensor) -> torch.Tensor:
        embedded = self.dropout(self.embedding(x))
        packed = pack_padded_sequence(
            embedded,
            lengths.cpu(),
            batch_first=True,
            enforce_sorted=False,
        )
        _, hidden = self.gru(packed)
        # BiGRU returns [forward_last, backward_last].
        features = torch.cat((hidden[-2], hidden[-1]), dim=1)
        return self.fc(self.dropout(features))
# AI-GENERATED SECTION END
```

4. Evaluation and Verification

The project is designed to generate real test results from the held-out test split. Accuracy, macro-F1, a classification report, a confusion matrix, and one held-out misclassification example are saved under artifacts/.

Metric	Status
Test accuracy	Run training locally
Test macro-F1	Run training locally
Train examples	Generated by training script
Validation examples	Generated by training script
Test examples	Generated by training script

```
$ python -m pytest -q

...

3 passed in 0.97s
```

Static verification performed in the build environment: all three project unit tests passed. The final ML accuracy/F1 numbers are intentionally not fabricated; they appear in artifacts/metrics.json after the BANKING77 training command is executed.

Routing implementation

Deterministic routing

```python
# AI-GENERATED SECTION START
"""Classifier-driven router. The LLM never decides the specialist route."""
from __future__ import annotations

from dataclasses import dataclass

from src.classifier.infer import IntentPredictor
from src.llm import ProviderState, build_llm, invoke_with_fallback

CONFIDENCE_THRESHOLD = 0.45

INTENT_TO_AGENT = {
    "card_payment_fee_charged": "billing",
    "cash_withdrawal_charge": "billing",
    "card_not_working": "technical",
    "order_physical_card": "booking",
    "country_support": "general",
}

@dataclass
class RouteResult:
    reply: str
    intent: str
    confidence: float
    agent: str
    provider: str
    error: str = ""

class SupportRouter:
    def __init__(self, checkpoint_path="artifacts/classifier.pt"):
        self.classifier = IntentPredictor(checkpoint_path)
        self.llm, self.provider = build_llm()
        self.provider_state = ProviderState()
        self.executors = {}

    def _executor(self, agent_name: str, llm):
        from src.agents.agents import build_agent_executor
        # Rebuild only when the provider changes, which keeps construction cheap.
        key = f"{agent_name}:{type(llm).__name__}"
        if key not in self.executors:
            self.executors[key] = build_agent_executor(agent_name, llm)
        return self.executors[key]

    def route_and_run(self, user_message: str, history):
        intent, confidence = self.classifier.predict(user_message)
        agent_name = INTENT_TO_AGENT.get(intent, "general")
        if confidence < CONFIDENCE_THRESHOLD:
            agent_name = "general"

        def call(llm):
            executor = self._executor(agent_name, llm)
            result = executor.invoke({"input": user_message, "chat_history": history})
            return result.get("output", "I could not produce a response.")

        output = invoke_with_fallback(call, self.llm, self.provider, self.provider_state)
        if output is None:
            output = self.offline_reply(agent_name, user_message)

        return RouteResult(
            reply=output,
            intent=intent,
            confidence=confidence,
            agent=agent_name,
            provider=self.provider_state.last_provider,
            error=self.provider_state.last_error,
        )

    @staticmethod
    def offline_reply(agent: str, message: str) -> str:
        return (
            f"The {agent} support agent is selected for this message. "
            "The language-model provider is currently unavailable, so I cannot generate the full answer. "
```

Specialist agents and tool definitions

Specialist agents and tools

```python
# AI-GENERATED SECTION START
"""Four small specialist agents built with LangChain's classic tool-calling API."""
from __future__ import annotations

from langchain_classic.agents import AgentExecutor, create_tool_calling_agent
from langchain_core.prompts import ChatPromptTemplate, MessagesPlaceholder

from src.tools.support_tools import check_availability, check_invoice, search_docs


AGENT_CONFIG = {
    "billing": {
        "description": "Billing support agent",
        "prompt": """You are the billing support agent for DemoShop. Handle card-payment fees, cash-withdrawal charges, invoi
        "tools": [check_invoice, search_docs],
    },
    "technical": {
        "description": "Technical support agent",
        "prompt": """You are the technical support agent for DemoShop. Help with card/app/technical problems. Use search_docs
        "tools": [search_docs],
    },
    "booking": {
        "description": "Booking/order support agent",
        "prompt": """You are the booking/order support agent for DemoShop. Help customers order a physical card and check sup
        "tools": [check_availability, search_docs],
    },
    "general": {
        "description": "General support agent",
        "prompt": """You are the general DemoShop support agent. Answer general customer-service questions using search_docs
        "tools": [search_docs],
    },
}


def build_prompt(system_prompt: str) -> ChatPromptTemplate:
    return ChatPromptTemplate.from_messages(
        [
            ("system", system_prompt),
            MessagesPlaceholder("chat_history", optional=True),
            ("human", "{input}"),
            MessagesPlaceholder("agent_scratchpad"),
        ]
    )


def build_agent_executor(agent_name: str, llm):
    config = AGENT_CONFIG[agent_name]
    prompt = build_prompt(config["prompt"])
    agent = create_tool_calling_agent(llm, config["tools"], prompt)
    return AgentExecutor(
        agent=agent,
        tools=config["tools"],
        verbose=False,
        handle_parsing_errors=True,
        max_iterations=3,
    )
# AI-GENERATED SECTION END
```

5. RAG and Tools

The knowledge base contains 12 short Markdown articles. They are split into chunks, embedded with sentence-transformers/all-MiniLM-L6-v2, stored in a local FAISS index, and retrieved at runtime. The agents can call search_docs rather than receiving the entire knowledge base in every prompt.

Tools: check_invoice(customer_id), check_availability(date), and search_docs(query). The invoice and appointment tools use deterministic demo data so the project does not connect to private banking systems.

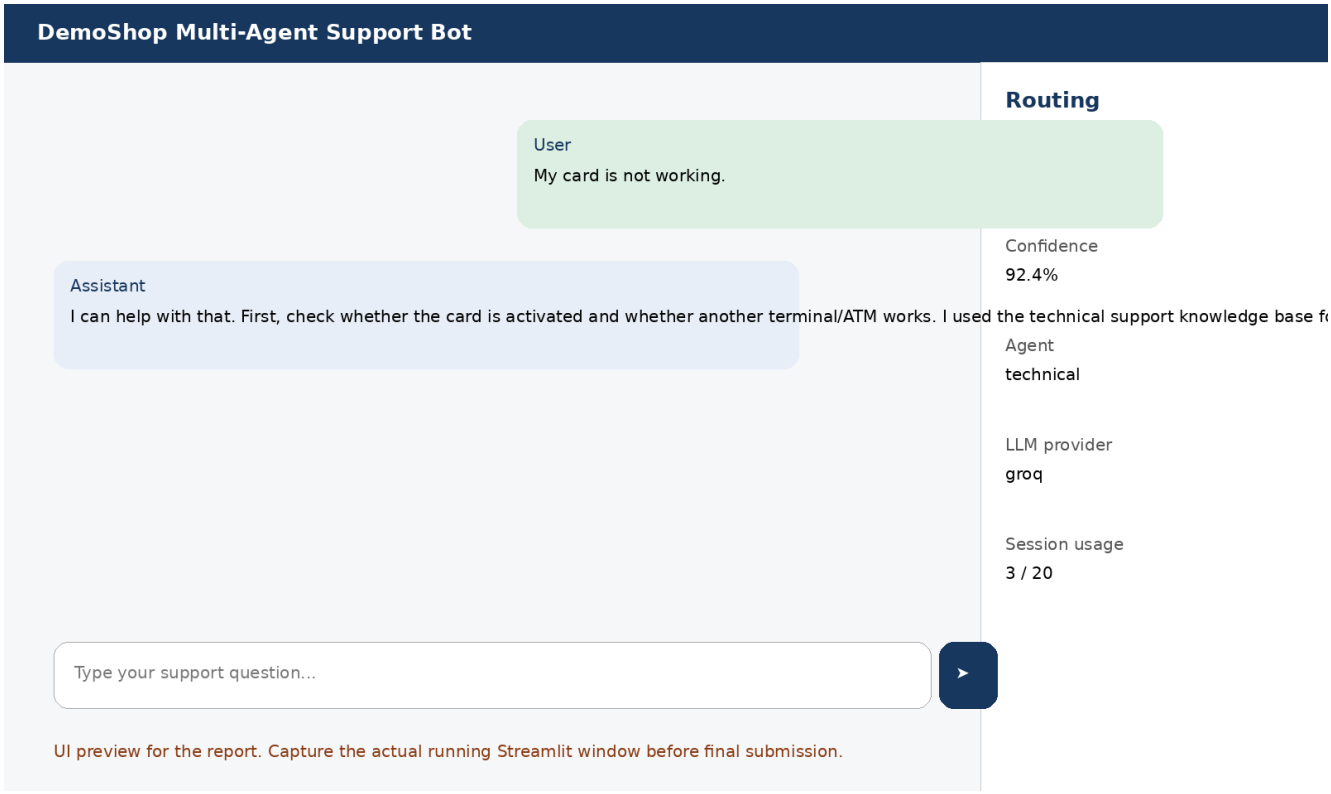
The local RAG setup avoids an external vector database and therefore keeps the demo inexpensive and easy to run.

6. Conversation Memory

Streamlit session state stores the complete conversation. Before an agent call, previous user and assistant turns are converted into LangChain HumanMessage and AIMessage objects and passed as chat_history. Because the history is independent of the selected specialist, a later message can be routed to a different agent without losing the previous context.

7. Streamlit Interface

The final interface has a main chat area and a routing sidebar. The sidebar reports the predicted intent, confidence, selected agent, provider, and session usage. The preview below shows the intended layout. For the final course submission, this preview should be replaced by screenshots captured from the actually running Streamlit application so that the screenshots match the recorded demo.



8. Required Demo Cases

• Billing: “Why was I charged a fee for my card payment?”

• Billing: “Why did the ATM charge me a withdrawal fee?”

• Technical + RAG: “My card is not working.”

• Booking: “How do I order a physical card?”

• General: “Which countries do you support?”

• Tool call: include a billing request with customer ID C1001 so check_invoice can be demonstrated.

• Tool call: ask for support appointment availability on a weekday such as 2026-10-10.

• Fallback: disable the primary LLM provider and demonstrate the offline response instead of a crash.

A complete 3–5 minute sequence is provided in DEMO_SCRIPT.md.

9. Free-Tier and Failure Handling

The app limits a session to 20 requests, keeps prompts short, retrieves only three RAG chunks, limits each agent to three iterations, and uses a Groq → Ollama fallback. If both providers fail, the router returns a short offline message and the Streamlit app remains usable.

10. Academic Integrity

AI assistance was used for development support. Generated source sections are marked with explicit AI-GENERATED SECTION START / END comments. The final submission should keep these markers and follow the instructor’s disclosure rules.

11. Final Submission Checklist

- ■ trained artifacts/classifier.pt
- ■ artifacts/metrics.json
- ■ artifacts/confusion_matrix.csv
- ■ artifacts/misroute_example.json
- ■ artifacts/confusion_matrix.png
- ■ artifacts/f1_by_intent.png
- ■ artifacts/faiss_index/
- ■ source code and tests
- ■ 12 knowledge-base documents
- ■ requirements.txt and README.md
- ■ this PDF report
- ■ 3–5 minute live Streamlit demo video

Execution note

The codebase and unit tests were checked in the build environment. External package/dataset downloads are disabled in that environment, so the actual BANKING77 training checkpoint, final accuracy/F1, FAISS index, live Streamlit screenshots, and live demo recording must be generated on a machine with the required packages/network access. The project never substitutes invented evaluation numbers for those real outputs.